



Open Architecture User Guide

V1.0

Table of Content

Open Architecture User Guide	0
Table of Content	1
Revision History	3
1. Introduction	4
1.1 Overview	4
1.2 Purpose	5
1.3 Abbreviations	5
2. Getting Started	6
2.1 System Requirements	6
2.2 Installation	6
3. Open Architecture	7
3.1 System Architecture	7
3.1.1 Block Diagram of Flow DSP	8
3.2 Function Description	9
3.2.1 Custom AO Builder	9
3.2.2 Custom AO main window	9
3.2.3 Custom AO list	11
3.2.4 AO property setting	12
3.2.5 Tuning parameter setting table	13
3.2.6 AO GUI	14
3.2.7 Configuration button	14
3.2.8 Status bar	14
4. Audio Object (AO) Creation	15
4.1 Audio Objects Introduction	15
4.2 AO Creation List	15
4.3 Creating Custom Audio Objects	16
5. Custom Algorithm Programming	19
5.1 Header file	21
5.2 Prepare function	21
5.3 Process function	22
5.4 Parameter handler function	23
6. Debugging and Logging	24
6.1 Enabling Debugging Terminal	24
6.2 Saving Logs to File	26
6.3 Visual studio – Attach to process	27
7. Core Components	29
7.1 URamping Module	29

7.2	UAudioFFT Module	29
7.3	UMath Module.....	30
7.4	Ustft Module	30
7.5	UFilterDesigner Module.....	30

Revision History

Date	Revision	Note	Author
2025.2.4	V1.0	Initial document	Clark Chen Tofu Tsai Vincent Lo

1. Introduction

1.1 Overview

What is **Open Architecture**? Before answering this question, let's first explain: What is **Flow DSP**? Flow DSP is a standalone software audio DSP solution designed to simplify the tuning of active speakers, headphones, and audio systems. It enables users to create customizable audio signal processing flows using predefined and user-defined **audio objects (AOs)** that can be dynamically interconnected. These signal flows can run on various hardware platforms or directly on a PC, offering flexibility in development and deployment.

Flow DSP consists of two parts. The **Flow Studio** that sits on a PC and the Flow Engine that sits on the target HW. The flows created using **Flow Studio** can be configured and executed directly on a supported processor. In order to successfully run the flows on the supported processor, the processor must hold an image of the Flow Engine.

Every audio object (AO) in Flow DSP is built upon the Flow Engine AO structure, which provides a well-defined interface and comprehensive debugging tools for seamless algorithm development. This structured approach simplifies the process for users, enabling them to create, modify, and optimize their own DSP algorithms with ease. **In this document, we define this architecture as Open Architecture (OA)—a flexible and extensible framework that empowers users to develop custom AOs and seamlessly integrate their proprietary algorithms into DSP workflows.** Unlike a closed system, where modifications are restricted, OA transforms Flow DSP into an adaptable platform, allowing users to expand its capabilities according to their unique needs.

With OA, users can:

- **Define and implement their own audio algorithms within custom AOs.**
- **Seamlessly integrate their custom AOs into existing Flow DSP processing chains.**
- **Utilize debugging tools to streamline development and optimization.**

This document serves as a comprehensive guide to creating custom AOs in Flow Studio and implementing custom algorithms within the Flow Engine AO structure. By providing a structured yet flexible environment, Open Architecture (OA) enables developers to efficiently design, test, and deploy their own DSP solutions. For a step-by-step introduction to AO development, refer to the Quick Start Guide.

1.2 Purpose

The **Open Architecture (OA)** provides a **simple, intuitive, and flexible development environment** for creating **user-defined audio objects (AOs)**. With this framework, users can independently develop and integrate their own **custom AOs**, eliminating the need to rely on the **Flow DSP team** for algorithm integration. This autonomy significantly **reduces development time** and accelerates innovation, allowing users to implement and test their DSP solutions with greater efficiency.

To further streamline development, the **AO Builder** application automatically generates a **well-structured project template**, ensuring a clear and organized foundation for AO creation. This allows developers to **focus entirely on algorithm development in C programming language**, without worrying about the complexities of project setup or system integration. By leveraging this streamlined workflow, users can rapidly prototype, test, and deploy their **custom DSP solutions**, enhancing both **productivity and flexibility** within the Flow DSP ecosystem.

1.3 Abbreviations

Term	Full Word
TYM	Tymphany
AO	Audio Object
DSP	Digital Signal Processing
OA	Open Architecture
GUI	Graphical User Interface

2. Getting Started

Most PCs will be able to run Flow Studio. Follow the instructions below to install Flow Studio and operation tutorial step by step, we can start building

2.1 System Requirements

- **OS Required**

- Windows 10/11

- **Software Tool Required**

- Visual Studio 2022 Community/Professional/Enterprise
- CMake 3.29

2.2 Installation

Download and install the latest version of Flow Studio by following steps:

1. Install the 3.x version of CMake and Visual Studio if it isn't installed on PC. The latest installation executable can be downloaded from their official website.
2. Go to Flow Studio official website to download the latest Flow Studio.
3. Open Flow Studio installer executable and follow the instructions to accept the license agreement to install the software. Please refer the document *Flow Studio User Guide* for detailed information about installation.

3. Open Architecture

3.1 System Architecture

The system architecture of Flow DSP is illustrated below. It is composed of 4 modules, including Flow Engine, Flow Studio, Audio Data Handler, and Sound Device.

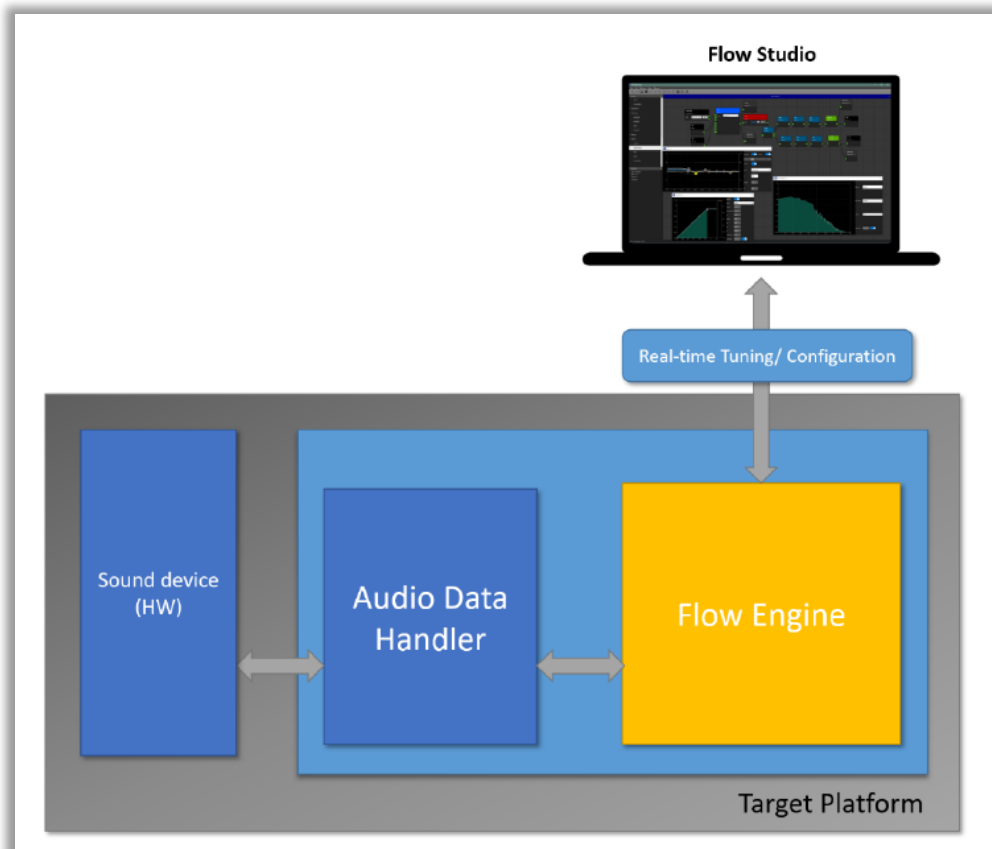


Figure 1. Typical architecture of Flow DSP system

- **Flow Engine** – handles all of the audio signal processing inside the target platform
- **Flow Studio** – Not handle audio processing directly; instead, it provides a user interface for configuring DSP applications through Flow Engine
- **Audio Data Handler** – serves as the audio driver on the target platform, facilitating audio data transmission and reception between the system's physical hardware and Flow Engine. The implementation of audio data handling varies across different platforms.
- **Sound Device** – refers to audio codecs or sound cards used in specific platforms.

3.1.1 Block Diagram of Flow DSP

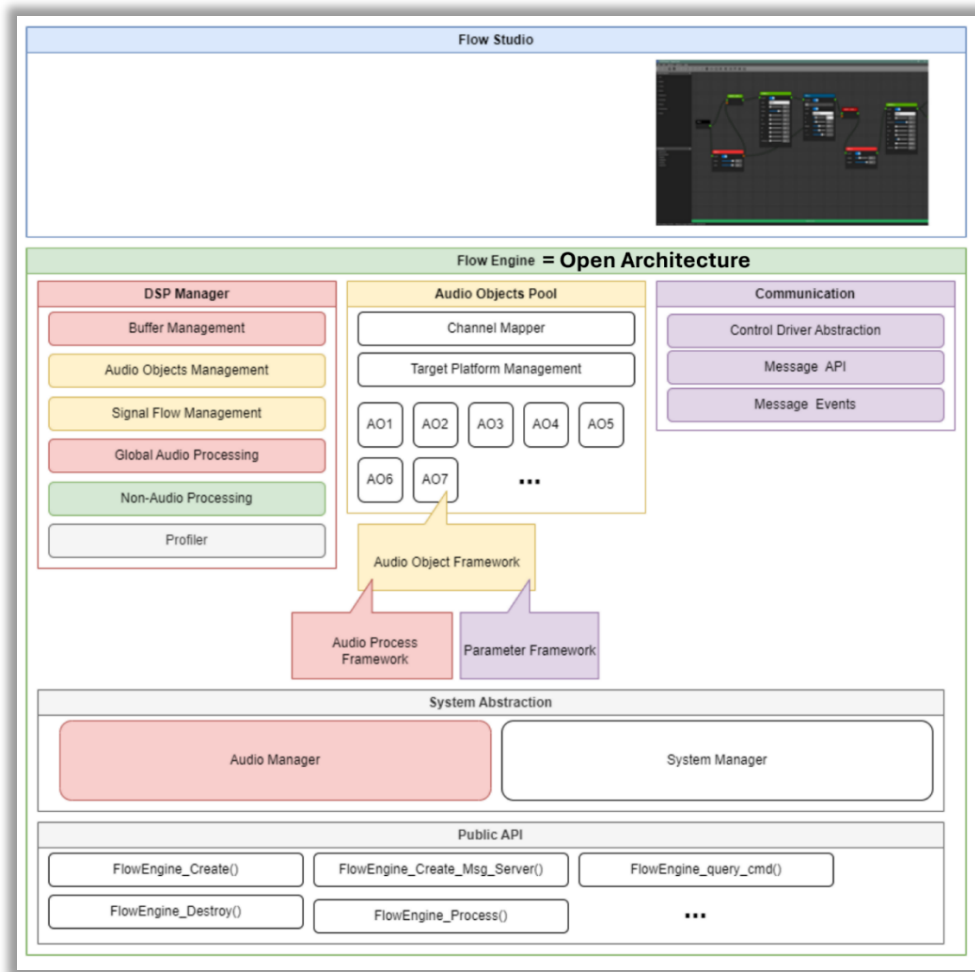


Figure 2. Block diagram of Flow DSP

Flow Engine is composed of several key components, each with specific functions and interrelationships, as outlined below:

- **Audio Objects Pool** – Manages the audio processing modules (algorithms) within Flow Engine. It provides a unified framework for handling audio processing and parameter control.
- **DSP Manager** – Utilizes configuration files generated by Flow Studio to dynamically create and initialize audio processing objects, set default parameters, establish interconnections, and integrate them into the global audio processing workflow within the engine.
- **Communication** – Facilitates the exchange of control messages between Flow Studio and the audio objects instantiated by the DSP Manager. Parameter control is event-driven.
- **Audio/System Manager** – Serves as an abstraction layer that isolates hardware-specific implementations from Flow Engine. These functionalities are not accessible through public APIs.
- **Public API Layer** – Provides exposed APIs that allow users to interact with the system. These APIs support both audio processing and control handling.

3.2 Function Description

3.2.1 Custom AO Builder

The audio object generation tools of open architecture is located in “Tools > Custom AO Builder”.

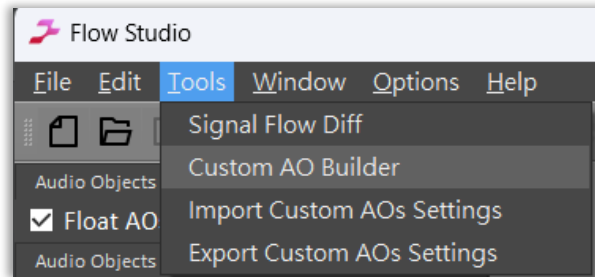


Figure 3. Custom AO Builder path

3.2.2 Custom AO main window

The **Open Architecture Builder** provides an intuitive interface for developers to create and manage custom Audio Objects (AOs) within the FlowDSP framework. This panel serves as the central workspace where all parameters and settings required for an AO are defined. Developers can specify algorithm behavior, input/output ports and parameters in a structured and user-friendly manner.

Once the configuration is complete, the tool automatically generates the necessary source files and a project structure compatible with standard IDEs, significantly reducing manual setup effort and integration time. The following section describes the layout and functionality of each component in the Open Architecture Builder, enabling developers to quickly get started with implementing their own audio algorithms and integrating them into Flow Studio.

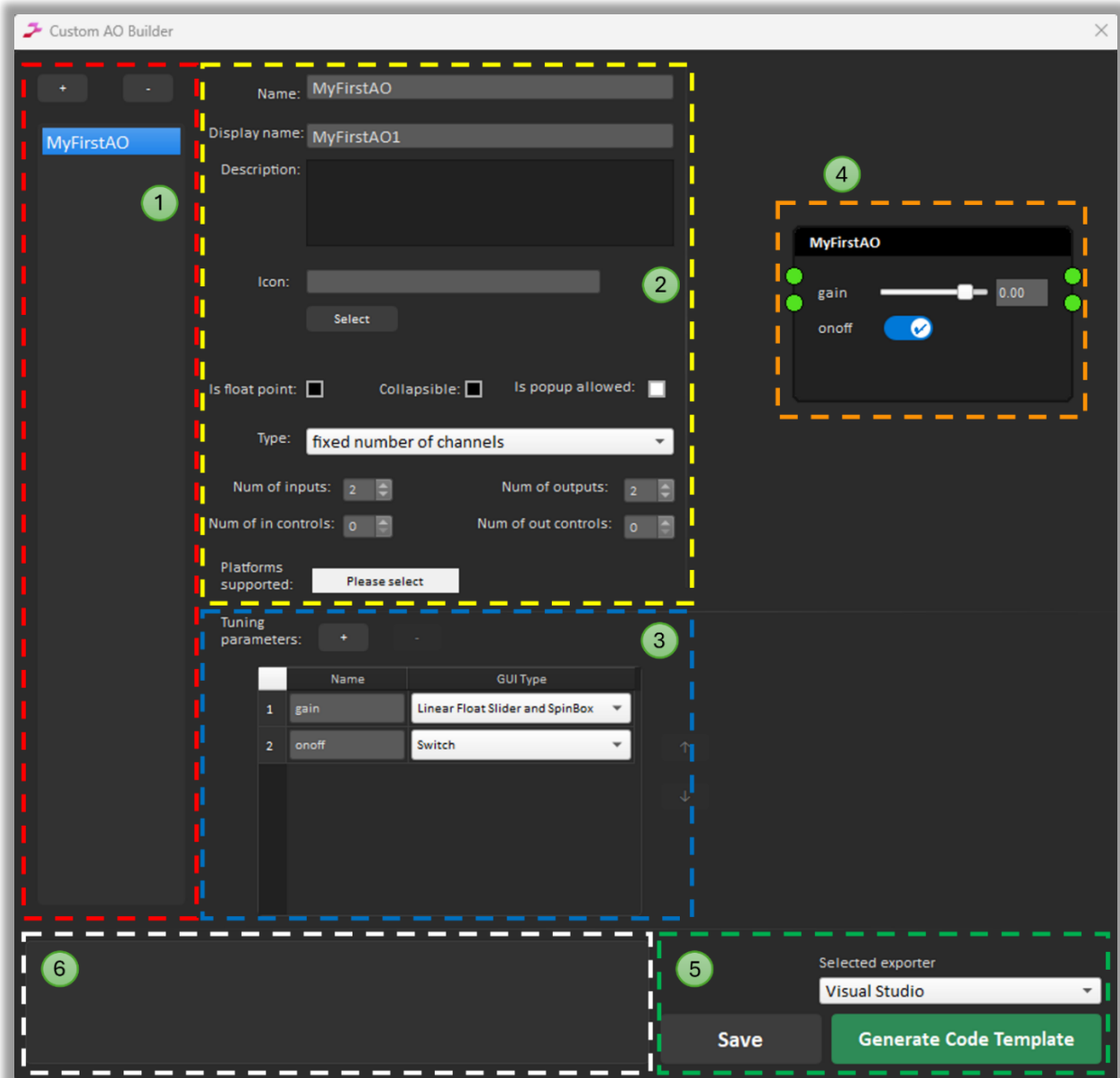
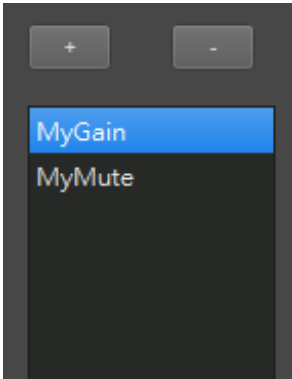


Figure 4. Custom AO Builder layout

1. Custom AO list (Red-Dotted Section)
2. AO property setting (Yellow-Dotted Section)
3. Tuning parameter setting table (Blue-Dotted Section)
4. AO GUI (Orange-Dotted Section)
5. Configure button (Green-Dotted Section)
6. Status bar (White-Dotted Section)

3.2.3 Custom AO list

Custom AO List is used to manage all user-defined Audio Objects (AOs). As algorithm development progresses, developers can add completed AOs to the **AO Creation List** to generate the corresponding source files and project structure.



Component	
Insert/Delete button	Add/Remove audio object module
AO list	List custom AO by order. Select AO to set up configuration setting or delete AO.

3.2.4 AO property setting

Name:

Display name:

Description:

Icon:

Select

Is float point: ☐
Collapsible: ☐
Is popup allowed: ☒

Type:

fixed number of channels

Num of inputs:

2

Num of outputs:

2

Num of in controls:

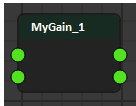
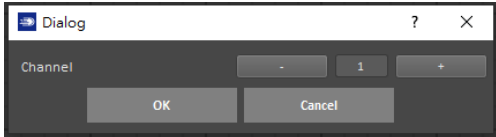
0

Num of out controls:

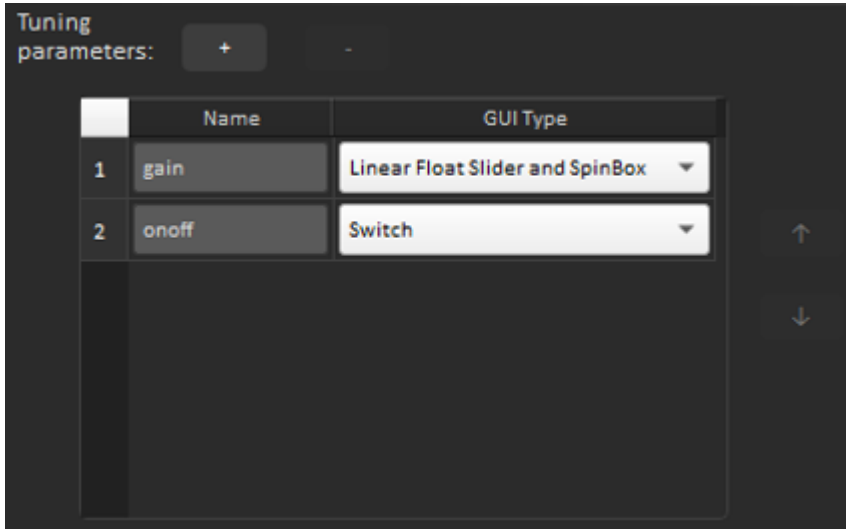
0

Platforms supported:

Please select

Component	
Name	AO name in Flow Engine code
Display name	The display name in Flow Studio audio object panel
Description	The AO information which displays in audio object panel. Ex: 
Icon	AO icon picture in audio object panel
Is float point	Setup the AO is float point or fixed point numeric representation format. Fill the block means float point, and empty block is fixed point.
Collapsible	Confirm whether AO module is collapsible. Filled block means collapsible. 
Is popup allowed	Confirm whether AO module has popup window display. Filled block means it has a popup window by double click module.
Type	Support fixed or dynamic in/out channel number. If selected fixed number, it will show below editors to setup in/out channel number. If select dynamic number, it will ask to setup in/out number when drag AO to design canvas. 
Num of inputs / outputs	Editor of input/output channel number
Num of controls inputs/outputs	Editor of control input/output port number
Platform supported	Selector of supported platform

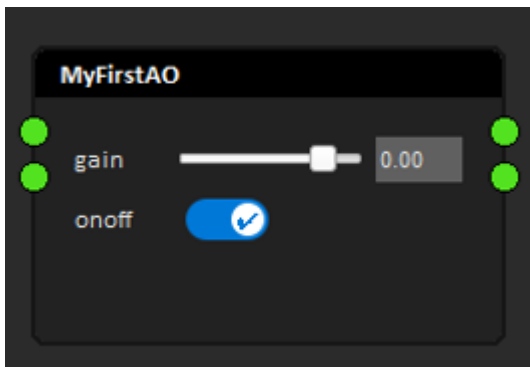
3.2.5 Tuning parameter setting table



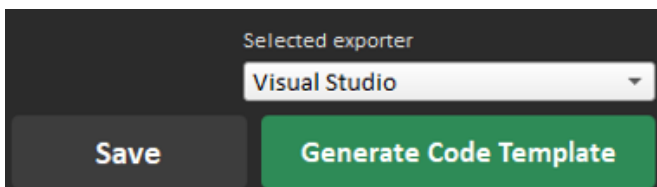
Component	
Insert/Delete button	Add/Remove tuning parameter
Name of parameter	Parameter name of Flow Studio.
GUI Type of parameter	Selection of parameter type. It has seven types, including <i>Switch</i> , <i>Menu</i> , <i>Linear Int Slider and SpinBox</i> , <i>Log Int Slider and SpinBox</i> , <i>Linear Float Slider and SpinBox</i> , <i>Log Float Slider and SpinBox</i> , <i>Label</i>
Parameters of parameter	It has different parameters of selected GUI type. The label_text is the parameter display name in AO module The screenshot shows the 'MyGain' module GUI. It has a title bar 'MyGain'. Below it, there are two parameters: 'gain' and 'onoff'. The 'gain' parameter is highlighted with a red box. The 'onoff' parameter is a toggle switch. The 'gain' parameter has a slider and a numeric display showing '0.00'. The 'onoff' parameter has a button with an 'X' icon.

3.2.6 AO GUI

The AO GUI displays graph interface of the audio object module in real-time.



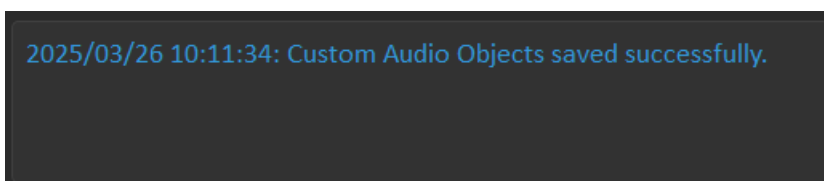
3.2.7 Configuration button



Component	
Cancel	Close the window and inquiry whether discard change
Generate Code Template	Generate code template in platform project Note: Currently, this code template is supported exclusively in Visual Studio.

3.2.8 Status bar

Display the status message for the configuration event.



4. Audio Object (AO) Creation

4.1 Audio Objects Introduction

The **audio object** represents an algorithm module managed by audio objects pool manager. It is responsible for handling audio processing and parameter control. It is defined as a **control unit** within Flow Studio. This design offers high flexibility, allowing each unit to be interconnected.

4.2 AO Creation List

The creation list stores all custom Audio Objects (AOs) implemented through the DSP Manager. It is defined in project/AppOA/AOCreationListCustom.c.

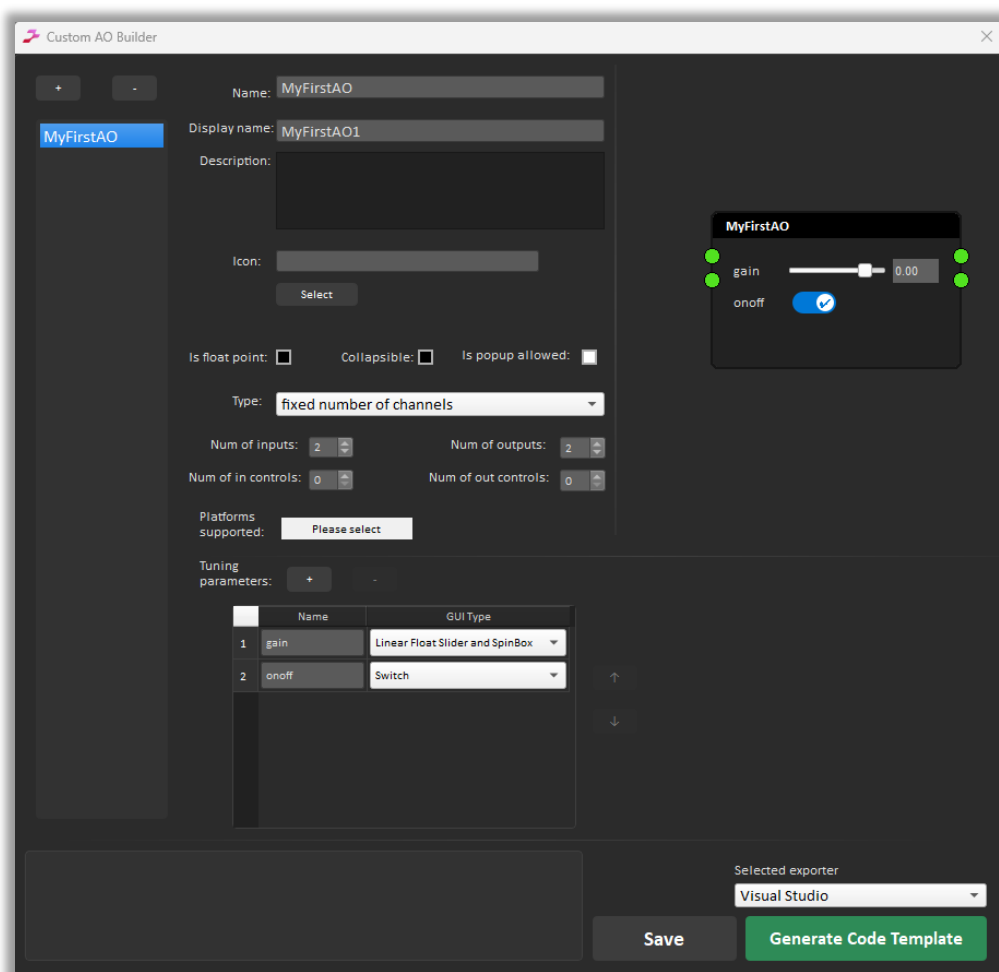
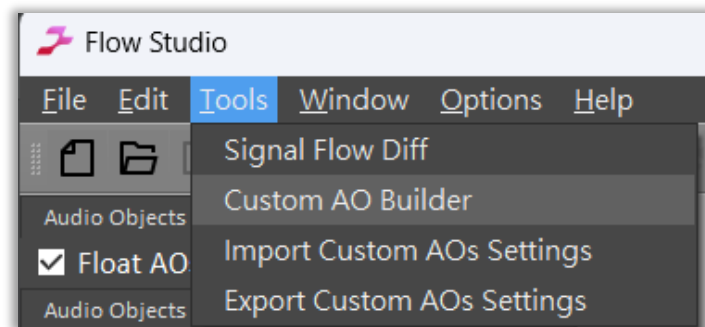


Figure 5. Custom AO Builder window

4.3 Creating Custom Audio Objects

This section outlines the procedure for generating a custom executable and embedding it into Flow Studio. Please follow the steps below.

1. Open FlowStudio.exe
2. Open “Custom AO Builder” from Tools menu



3. Enter algorithm data information into Custom AO Builder. Then press “Generate Code Template” button to create Visual Studio solution (.sln) which will be located in <Flow Studio installed Path>\project” folder path (see figure 5 from the previous page). The status message will be shown in status bar as below

“Generating code templates...”
“Code templates generated successfully.”
“Generating Visual Studio solution...”
“Visual Studio solution generated successfully.”
4. Open Visual Studio solution file to program custom AO. File locate in “<Code generated Path>\project”. Refer to chapter 5 “Custom Algorithm Programming” for detailed programming guide.
5. After complete code programming, select “Release” and “x64” solution configuration in Visual Studio. (see figure 6 in the next page)
6. Build project and generate customer Flow Engine executable named “AppOA.exe” which located in “<Code generated Path>\project\x64\Release” folder path (see figure 7 in the next page).
7. Create a custom signal routing project that includes new AO. Then, select generated AppOA.exe as the Engine path in Connection Setting to reference the new executable integrating the new AO (see figure 8~9 in the next page).
8. Press “Connect To Engine” in Setting window to activate audio processing successfully. The green status bar means connecting successfully. (see figure 9~12 in the next page).

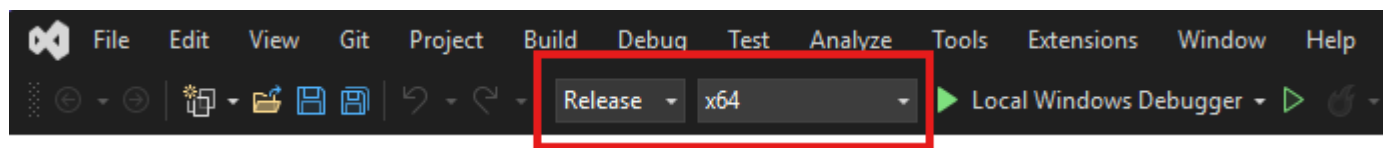


Figure 6. Set up solution configuration

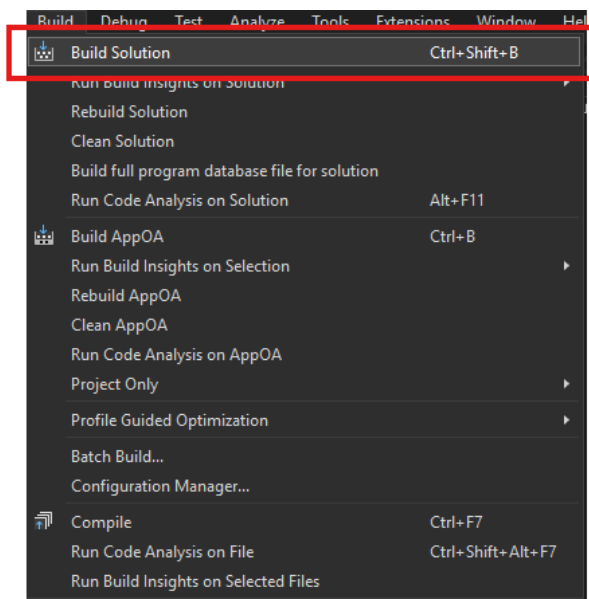


Figure 7. Build solution to generate AppOA.exe

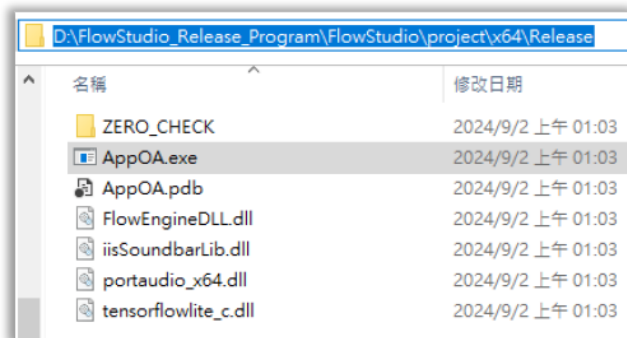


Figure 8. File path of AppOA.exe

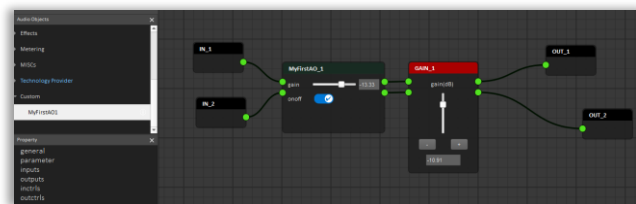


Figure 9. Audio routing path with customer A0

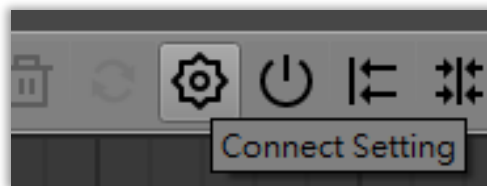
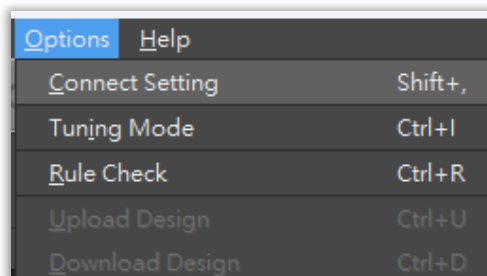


Figure 10. Select Connection Setting from menu or toolbar



Figure 11. Select AppOA.exe to execute new A0

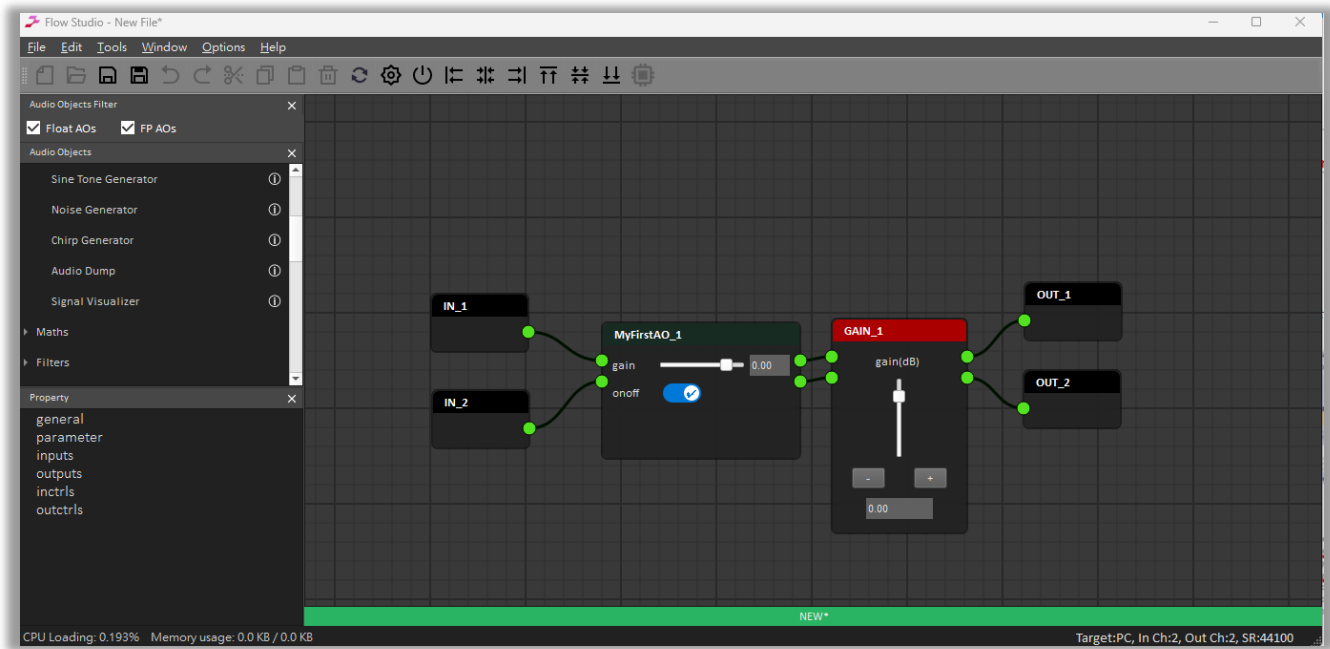


Figure 12. Connect audio processing with new AO successfully

5. Custom Algorithm Programming

The following illustrates the **open architecture execution process**:

- **Green blocks** represent Flow DSP tasks, including UI interface generation for customization, IDE project creation for a specific target, and algorithm integration into Flow DSP.
- **Orange blocks** represent developer tasks, which involve programming algorithms within the Flow DSP architecture using a target-specific IDE and building the executable or library.

The Custom AO Builder automatically handles most algorithm initialization. Developers only need to declare the variables, allocate memory in the preparation function, and implement the audio processing and parameter handler functions.

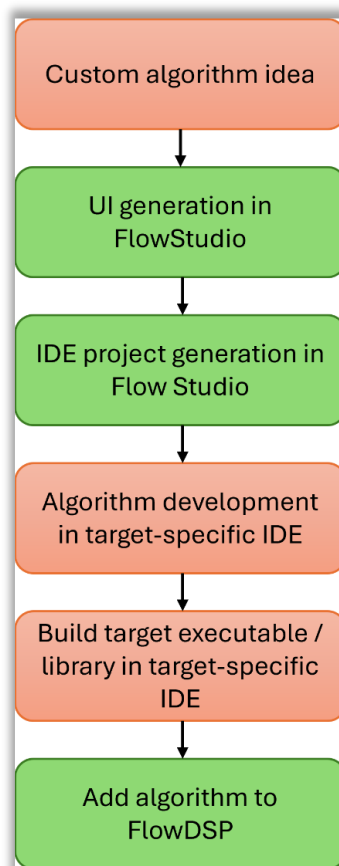


Figure 13. Open Architecture Execute Process

There are comments to define the Flow DSP auto-generated code region and the user code region. It is recommended not to modify the auto-generated code to prevent it from being overwritten when regenerating the code template. Please implement custom code within the user code region. (Fig. 14~15)

```

OPT_OS void CMyFirstAO_Delete(CMyFirstAO* obj)
{
    if(obj->cao_base->_isPrepared)
    {
        CMyFirstAO_Release(obj->cao_base);
    }

    // BEGIN: AUTO GEN FREE PARAM CODE
    // free parameter memory
    Delete(CParam, obj->_pgain);
    Delete(CParam, obj->_ponoff);

    // END: AUTO GEN FREE PARAM CODE

    Delete(CAudioObject, obj->cao_base);
}

```

Figure 14. Auto-generated code region example

```

// Your audio-processing code goes here!
// For more details, see the help from example project
// Right now we are not producing any data, in which case we need to clear the buffer
// (to prevent the output of random noise)
/* USER CODE BEGIN Process */

for (int channel = 0; channel < ch; channel++)
{
    URamping_Ramp(This->_gainCoeef[channel]);

    float* inputs = ins->GetChannelData(ins, channel);
    float* outputs = outs->GetChannelData(outs, channel);

    mult(outputs, inputs, URamping_GetValue(This->_gainCoeef[0]), me->GetFrameSize(me));
}
/* USER CODE END Process */

```

Figure 15. User code region example

5.1 Header file

Before starting to program algorithms, declare public variables, like audio data buffer or common variables in header file.

```
struct _CMyFirstAO
{
    CAudioObject* cao_base;

    // declare variable for audio processing
    int _channels;

    /* USER CODE BEGIN VarDeclare */
    float _myGain;
    URamping** _gainCoef;
    /* USER CODE END VarDeclare */

    // BEGIN: AUTO GEN DECLARE PARAM CODE
    // declare parameter CParam which display on Audio Object content
    // example, CParam* _pParam1;
    CParam* _pgain;
    CParam* _ponoff;

    // END: AUTO GEN DECLARE PARAM CODE
};
```

Custom-defined variable

Auto-generated variable and parameter

Figure 16. Variable definition in header file

5.2 Prepare function

Initialize variable values and allocate memory for the variable array in the Prepare function. These variables are associated with the frame size. Initialization of other variable values can be configured in the AO New function.

```
OPT_OS int CMyFirstAO_Prepare(CAudioObject* me, float sampleRate, int frameSize)
{
    CMyFirstAO* This = me->_DerivedObj;

    me->SetSampleRate(me, sampleRate);
    me->SetFrameSize(me, frameSize);

    // This function will be called when the audio device is started, or when
    // its settings (i.e. sample rate, block size, etc) are changed.
    // You can use this function to initialise any resources you might need,
    // For more details, see the help from example project
    /* USER CODE BEGIN MemoryAllocate */
    //Allocate gain ramping object
    This->_gainCoef = (URamping**)flow_calloc(This->_channels, sizeof(URamping*));
    for (int i = 0; i < This->_channels; ++i)
    {
        This->_gainCoef[i] = URamping_New(1.0f, 50.0f);
        This->_gainCoef[i]->Prepare(This->_gainCoef[i], sampleRate, frameSize);
    }
    /* USER CODE END MemoryAllocate */
    me->_isPrepared = 1;

    me->UpdateAllParamsToDefault(me);
    return 0;
}
```

Figure 17. Variables initialization and array allocation in Prepare function

5.3 Process function

To program the process function for algorithmic audio processing, the default behavior bypasses the input to the output. Below is an example of gain control

```
void CMyFirstAO_Process(CAudioObject* me, const void* in, void* out, int frameSize)
{
    AudioBuffer* ins = (AudioBuffer*)in;
    AudioBuffer* outs = (AudioBuffer*)out;;
    CMyFirstAO* This = me->DerivedObj;
    int ch = This->_channels;

    // Your audio-processing code goes here!
    // For more details, see the help from example project
    // Right now we are not producing any data, in which case we need to clear the buffer
    // (to prevent the output of random noise)
    /* USER CODE BEGIN Process */

    for (int channel = 0; channel < ch; channel++)
    {
        URamping_Ramp(This->_gainCoef[channel]);

        float* inputs = ins->GetChannelData(ins, channel);
        float* outputs = outs->GetChannelData(outs, channel);

        mult(outputs, inputs, URamping_GetValue(This->_gainCoef[0]), me->GetFrameSize(me));
    }
    /* USER CODE END Process */
}
```

URamp gain control
thru sample buffer

Figure 18. Process function code example

5.4 Parameter handler function

Implement the parameter handler to compute equations based on parameter configurations. Below is an example of gain calculation converting decibels (dB) to a linear scale.

```
OPT_OS void CMYFirstAO_MyFirstAOChangeHandler(CParam* param, float newValue, int row, int column)
{
    int ch = column;
    CMYFirstAO* This = param->Owner;

    // This handler function handle the parameter change action, needs to register handler in the beginning of file
    // For more details, see the help from example project
    /* USER CODE BEGIN ParamHandler */
    float gainVal = This->pgain->GetValue(This->pgain, row, column);
    float onoffVal = This->ponoff->GetValue(This->ponoff, row, column);
    float gainLinear = powf(10.0f, (gainVal * 0.05f));

    if (gainVal < This->pgain->GetMinValue(This->pgain)) //if gain value lower than the minima value, set to mute
    {
        This->gainCoef[ch]->SetTargetValue(This->gainCoef[ch], 0.0f, 0);
    }
    else
    {
        if (onoffVal == 1.0f)
        {
            This->gainCoef[ch]->SetTargetValue(This->gainCoef[ch], gainLinear, 0);
        }
        else if (onoffVal == 0.0f)
        {
            This->gainCoef[ch]->SetTargetValue(This->gainCoef[ch], 1.0f, 0);
        }
    }
}
/* USER CODE END ParamHandler */
}
```

Figure 19. Parameter handler function code example

6. Debugging and Logging

The open architecture supports a debugging method that involves running the Flow Studio project with the debugging terminal enabled for real-time logging, displaying log outputs from the open architecture C code.

6.1 Enabling Debugging Terminal

When running the custom **AppOA.exe** executable with a custom AO, a debug selection option is available for developers. This option allows log outputs from the Visual Studio C code to be displayed. Follow the instructions below to enable real-time debugging mode in the terminal:

1. Open the connection setting (See Figure 10)
2. Select AppOA.exe from the <Code generated Path>\project\x64\Release path to execute new AO. (See Figure 11). It will pop-up Debug option.
3. Check the debug option in the lower right corner of the Setting window. (Fig. 20)
4. Press “**Connect to Engine**” to successfully activate audio processing. Real-time terminal logging will be displayed for debugging. (Fig. 21)



Figure 20. Check Debug option

```

[ FLOW ] ----- Total 109 algorithms available -----
[ FLOW ] ----- parsing flw config ... -----
FLOW Studio check point: valid input file
This->_gainCoef[0]: 0.0000
This->_gainCoef[1]: 0.0000
This->_gainCoef[0]: 0.0000
This->_gainCoef[1]: 0.0000
[ FLOW ] param: onoff, 1.000000, 0, 0
This->_gainCoef[0]: 0.0000
[ FLOW ] param: gain, 0.000000, 0, 0
This->_gainCoef[0]: 0.0000
[ FLOW ] -- preparing buffers...
[ FLOW ] ----- processing order -----
[ FLOW ] 0, IN_1
[ FLOW ] 1, IN_2
[ FLOW ] 2, MyFirstAO_1
[ FLOW ] 3, OUT_1
[ FLOW ] 4, OUT_2
[ FLOW ] -- start wiring...
[ FLOW ] wiring success
Devices:
0: (input) Microsoft Ĺ - Input (MME)
1: (input) JRC (Realtek(R) Audio) (MME) --default--
2: (output) Microsoft Ĺ - Output (MME)
3: (output) z (Realtek(R) Audio) (MME) --default--
4: (duplex) Realtek ASIO (ASIO)
[ FLOW ] *** Max Input Channels: 2
[ FLOW ] *** Max Output Channels: 2
[ FLOW ] ***** input latency: 0.012880
[ FLOW ] ***** output latency: 0.090159
[ FLOW ] Socket Server initialization succeed!
FLOW Studio check point: socket server init succeed
FLOW Studio check point: socket created succeed
[ FLOW ] Maxfd: 1160 Linstener on port: 54010, waiting for connections...
[ FLOW ] ----- MyFirstAO_1 -----
[ FLOW ] gain                                >> printf log
This->_gainCoef[0]: 0.0000
[ FLOW ] -0.640000                        >> LOGC log
[ FLOW ] ----- MyFirstAO_1 -----
[ FLOW ] gain
This->_gainCoef[0]: 0.0000
[ FLOW ] -1.920000
[ FLOW ] ----- MyFirstAO_1 -----
[ FLOW ] gain
This->_gainCoef[0]: 0.0000

```

Figure 21. Real-time terminal logging

6.2 Saving Logs to File

In addition to real-time terminal logging, logs are saved as text files in the <Code Generated Path>\project\x64\Release directory. The log file is named based on the date and a new log file is appended daily.

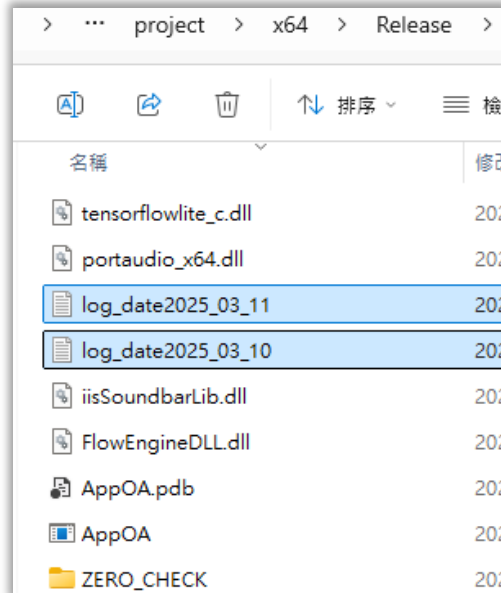


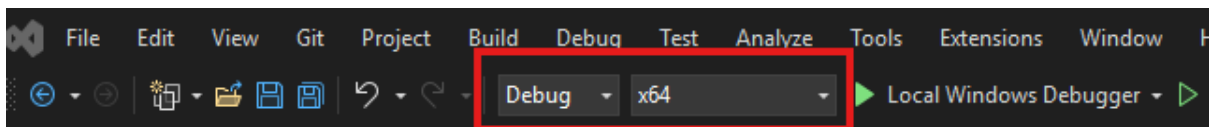
Figure 22. log file appended daily

6.3 Visual studio – Attach to process

In addition to using log output to check the results of C code for debugging purposes, developers working in Visual Studio can take advantage of a real time debugging approach. When Flow Studio runs the custom AppOA.exe, you can use Visual Studio's "Attach to Process" feature to attach to the running process. This allows you to set breakpoints and perform step-by-step debugging directly in your C code.

⚠ Note: This feature requires AppOA.exe to be built in Debug mode in order to properly attach and enable breakpoints. This method provides a more intuitive way to observe the execution flow and variable states in real time, making it easier to identify issues and fine-tune algorithms.

1. In Visual Studio, select the Debug x64 configuration and build the project.



2. Launch Custom AO using Flow Studio. (Fig. 23)
3. Go back to Visual Studio, then click "Debug" > "Attach to Process..." from the top menu. In the dialog box, find and select AppOA.exe, then click Attach. (Fig. 24)
4. You can now set breakpoints in your C code and perform real-time debugging to monitor variables and execution flow. (Fig. 25)

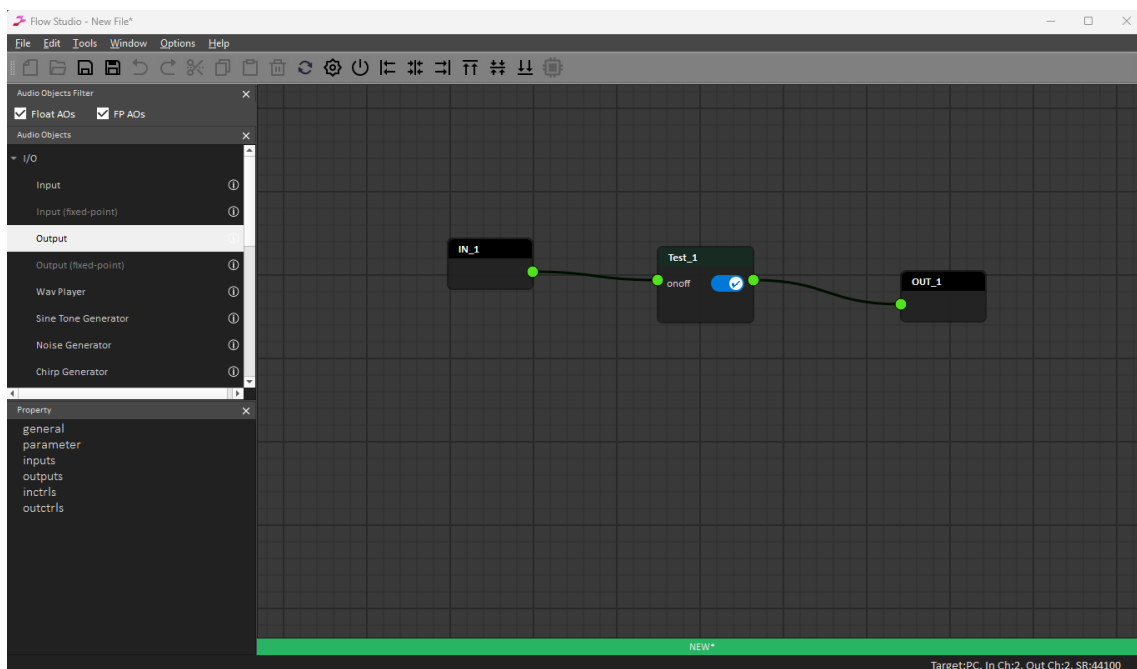


Fig. 23 Launch Custom AO using Flow Studio

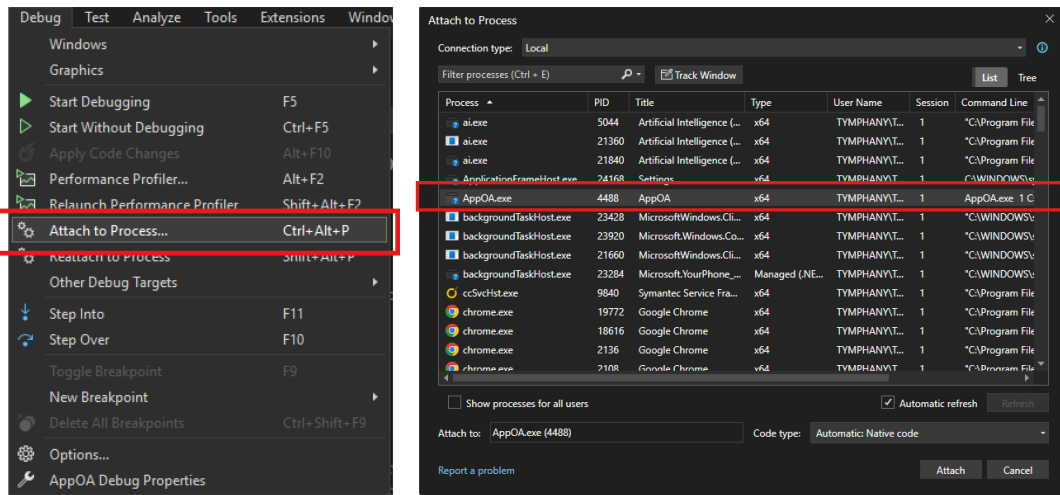


Fig. 24 Attach AppOA.exe to the running process for debugging

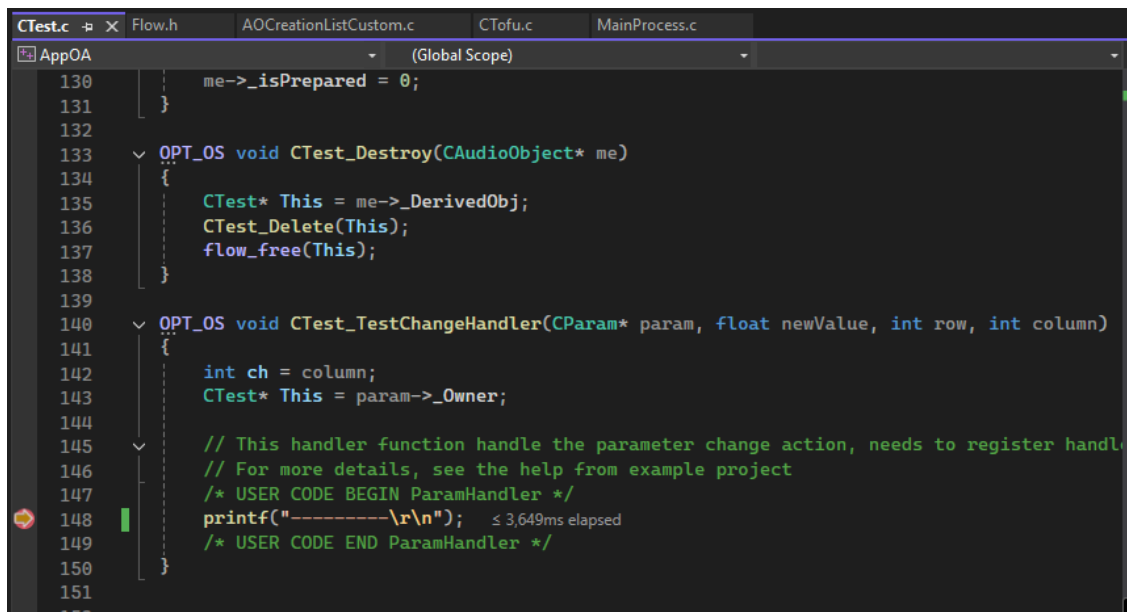


Fig. 25 Launch Custom AO using Flow Studio

7. Core Components

We provide core components for customization to accelerate audio processing development. This enables developers to leverage existing AOs within Flow Engine.

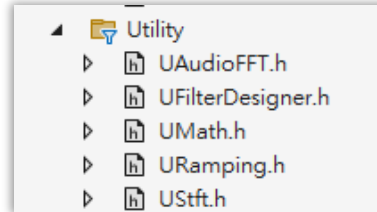


Figure 26. Core components headers store in Utility file of Visual Studio solution

7.1 URamping Module

The URamping module is a ramping gain object used for audio processing to avoid clipping noise caused by rapid gain change. It declares the functions used to create, delete and manipulate URamping objects.

```
1. URamping* ramp = URamping_New(0.0f, 1000.0f);
2. ramp->Prepare(ramp, 44100.0f, 512);
3. ramp->SetTargetValue(ramp, 1.0f, 0);
4. while (ramp->_currentVal != ramp->_targetVal) {
5.     URamping_Ramp(ramp);
6.     printf("Current Value: %f\n", URamping_GetValue(ramp));
7. }
8. URamping_Delete(ramp);
```

Figure 27. URamping example usage

7.2 UAudioFFT Module

The UAudioFFT structure is used to perform FFT and IFFT operations on audio data. It supports different setups based on the platform and configuration.

```
1. UAudioFFT* fftInstance = UAudioFFT_New(nullptr);
2. UAudioFFT_Init(fftInstance, 1024);
3.
4. float data[1024] = { /* input data */ };
5. float re[512], im[512];
6. fft(fftInstance, data, re, im);
7.
8. float output[1024];
9. ifft(fftInstance, output, re, im);
10.
11. (fftInstance);
```

Figure 28. UAudioFFT example usage

7.3 UMath Module

This file involves various mathematical calculation formulas to customer to accelerate audio processing development.

7.4 Ustft Module

This file contains the definition of the UStft structure, which is used for performing Short-Time Fourier Transform (STFT) and Inverse Short-Time Fourier Transform (ISTFT) operations. It also includes function declarations for creating, initializing, processing, and destroying. UStft objects, as well as various utility functions for manipulating FFT and IFFT data.

The UStft structure supports different window types and overlap-add algorithms, and it can be configured for various FFT frame lengths and hop sizes. The implementation uses the PFFFT library for FFT computations.

7.5 UFilterDesigner Module

The file provides various enumerations for filter types, crossover types, and adaptive filter types. It also declares the FilterDesign and FilterDesign_Fixed functions for designing filters.

```
1. sBiquadCoefs coefs;  
2. FilterDesign(1000.0f, 0.0f, 0.707f, 44100.0f, kLowPass2, &coefs);
```

Figure 29. UFilterDesigner example usage